

Score-Based Diffusion Model

An introduction

Yukinari Hisaki

Contents

1. Why Diffusion?
2. Score Matching Basics
3. Introduction of Diffusion
4. Step 1: Specify the SDE (VP SDE)
5. Step 2: Train the Score (Score Matching)
6. Step 3: Generate (Reverse ODE)
7. Summary: The Diffusion Framework
8. Controllable Generation: Guiding the SDE

- 9.** Advanced Capabilities Enabled by the ODE Formulation
- 10.** Accelerating Generation: DPM-Solver++

Why Diffusion?

- **The Goal:** How should we model complex human driving behavior using machine learning?
 - **The Naive Approach (Regression):** The simplest method is to learn a deterministic function $\tau = f_{\theta(c)}$ that maps the context c to a single future trajectory τ .
 - Input c : Ego state, other vehicles, map data, etc.
 - Output τ : Sequence of states $\{x_t, x_{t+1}, \dots, x_T\}$.
 - **The Limitation: However**, human behavior is inherently **multimodal** and uncertain.
 - In ambiguous scenarios, there are multiple valid maneuvers.
- ⇒ We need a **Generative Model** that learns the full distribution $p(\tau \mid c)$, not just a single path.

The central goal is to estimate an unknown data distribution $q(x)$ from samples $x \sim q(x)$.

- **The Standard Approach (e.g., VAEs):** Introduce a latent variable $z \sim \mathcal{N}(0, I)$ and map it to data via a **single** probabilistic decoding step $p_\theta(x|z)$.

$$z \xrightarrow{p_\theta(x|z)} x$$

- **The Limitation:** Learning the complex data distribution $q(x)$ via a “**single shot**” $p_\theta(x|z)$ is challenging. The gap between the simple prior and the complex data manifold is too large.
- **The Solution (Diffusion Models):** Decompose the generation into **multiple gradual transformations**. We refine the sample step-by-step. $x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$

$$x_T \sim \mathcal{N}(\cdot \mid 0, I), x_0 \sim q(x_0)$$

The idea of “Diffusion Model” can be mathematically formulated in three main ways:

1. Denoising Diffusion Probabilistic Models (DDPM) [1], [2]

- Formulated as discrete Markov chains.
- Optimized via Variational Lower Bound (ELBO).
- Key idea: Learn to reverse the noise addition step-by-step.

2. Score-based Generative Models [3]

- Formulated via Stochastic Differential Equations (SDEs).
- Key idea: Learn the **score function** $\nabla_x \log p_t(x)$ (direction to higher density).

3. Flow Matching [4]

- A more general framework based on Ordinary Differential Equations (ODEs).
- Key idea: Directly regress a **vector field** $v_t(x)$ to transport probability paths.

Focus of this talk: We will focus on **2. Score Matching**.

Score Matching Basics

The Goal: Given a dataset of samples $\{x_1, \dots, x_N\}$ from an unknown distribution $q(x)$, we want to learn a model $p_\theta(x)$ that approximates it.

- **The Modeling Challenge:** Flexible probability models are often defined up to a normalization constant (Energy-Based Models):

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}, \quad Z_\theta = \int \exp(-E_\theta(x)) \, dx$$

- **The Intractable Partition Function:** To train via Maximum Likelihood Estimation (MLE), we need to maximize $\log p_\theta(x)$. However, calculating Z_θ (the integral over high-dimensional space) is computationally impossible.

$$\log p_\theta(x) = -E_\theta(x) - \underbrace{\log Z_\theta}_{\text{Intractable!}}$$

- **The Solution (Differentiation):** If we take the gradient with respect to input x , the constant Z_θ vanishes!

$$\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x) - \underbrace{\nabla_x \log Z_\theta}_0 = -\nabla_x E_\theta(x)$$

By modeling the gradient of the log-density instead of the density itself, we bypass the normalization problem entirely.

Definition 0.1 (The Score Function): For a probability density $p(x)$, the score is defined as:

$$s(x) := \nabla_x \log p(x).$$

- **Intuition: A Vector Field**

- ▶ The score is a vector field pointing in the direction of **steepest ascent** (higher data density).
- ▶ Crucially, it creates a valid training target without ever needing to compute Z_θ .

- **The Objective (Score Matching):** We train a neural network $s_{\theta(x)}$ to match the data score:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{x \sim q} [\| s_\theta(x) - \nabla_x \log q(x) \|_2^2]$$

Problem: The true score $\nabla_x \log q(x)$ is intractable.

Theorem 0.1 (Equivalence of Explicit and Denoising Score Matching): Let $q(x)$ be the data distribution and $q(\tilde{x}|x)$ be a perturbation kernel. Define the smoothed marginal distribution as $q(\tilde{x}) = \int q(\tilde{x}|x)q(x) \, dx$.

Optimizing the explicit score matching objective for the smoothed marginal,

$$\mathcal{L}_{\text{ESM}}(\theta) = \mathbb{E}_{q(\tilde{x})} [\| s_{\theta}(\tilde{x}) - \nabla_{\tilde{x}} \log q(\tilde{x}) \|_2^2],$$

is equivalent to optimizing the Denoising Score Matching objective up to a constant C independent of θ :

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{q(x)q(\tilde{x}|x)} [\| s_{\theta}(\tilde{x}) - \nabla_{\tilde{x}} \log q(\tilde{x}|x) \|_2^2].$$

$$\Rightarrow \mathcal{L}_{\text{ESM}}(\theta) = \mathcal{L}_{\text{DSM}}(\theta) + C$$

The Core Intuition: Calculating the marginal score $\nabla_{\tilde{x}} \log q(\tilde{x})$ is intractable because $q(\tilde{x}) = \int q(\tilde{x}|x)q(x) \mathrm{d}x$ depends on the unknown data distribution $q(x)$. This theorem allows us to use $\mathcal{L}_{\text{DSM}}$ instead, which relies on $q(\tilde{x}|x)$. **Crucially**, the perturbation kernel $q(\tilde{x}|x)$ is independent of the unknown data $q(x)$. Since we design this kernel ourselves, its score $\nabla_{\tilde{x}} \log q(\tilde{x}|x)$ is analytically computable.

Choice of Conditional Distribution: Typically, we choose a **Gaussian with small variance** for $q(\tilde{x}|x)$ (e.g., $\mathcal{N}(\tilde{x}; x, \sigma^2 I)$). This makes the conditional score analytically tractable:

$$\nabla_{\tilde{x}} \log q(\tilde{x}|x) = -\frac{\tilde{x} - x}{\sigma^2}$$

and allows $q(\tilde{x})$ to approximate the data distribution $q(x)$ closely.

Once we have the score function $s_\theta(x) \approx \nabla_x \log q(x)$, how do we generate samples?

- **Langevin Dynamics:** We initialize x_0 **arbitrarily** and iteratively update it using the score:

$$x_{t+1} \leftarrow x_t + \underbrace{\varepsilon_t s_\theta(x_t)}_{\text{Gradient Ascent}} + \underbrace{\sqrt{2\varepsilon_t} z_t}_{\text{Random Noise}}$$

where $z_t \sim \mathcal{N}(0, I)$ and ε is the step size.

- **Mechanism:**
 1. **Gradient Term:** Pushes x towards high-density regions (modes).
 2. **Noise Term:** Prevents collapsing to a single mode, allowing exploration of the full distribution.

$\Rightarrow$ As $t \rightarrow \infty$ and $\varepsilon_t \rightarrow 0$, x_t converges to a sample from $q(x)$.

Theory is sound, but training on raw data $x \sim q(x)$ often fails in practice.

1. The Manifold Hypothesis (Blindness)

- High-dimensional data concentrates on a low-dimensional manifold.
- Outside this manifold, $q(x) \approx 0$.
- The score $\nabla_x \log q(x)$ is **undefined** or numerically unstable in the ambient space.

2. Inaccurate Estimation in Low-Density Regions

- The loss $\mathbb{E}_{x \sim q}$ only minimizes error **where data exists**.
- In low-density regions (most of the space), the model $s_{\theta(x)}$ is untrained.
- **Result:** When sampling starts from random noise (low density), the gradients are inaccurate, leading to garbage generation.

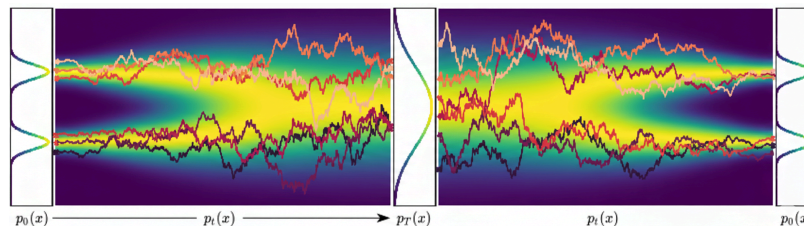
3. Slow Mixing of Langevin Dynamics

- If the data distribution has disjoint modes (isolated “islands” of data), the score in between is near zero.
- The sampler gets stuck in one mode and cannot cross the gap to others.

Introduction of Diffusion

How do we avoid to calculate the score in the region where the probability is defined?

- **The Strategy: A Continuous Path** Instead of modeling q directly, consider a continuous sequence of distributions p_t for $t \in [0, T]$ that connects:
 - $p_0 \approx q$ (The complex data distribution).
 - $p_T \approx \mathcal{N}(0, I)$ (A tractable noise distribution defined everywhere).
- **Two Key Challenges:**
 1. **Training:** How do we learn the time-dependent score $s_t(x) = \nabla_x \log p_t(x)$ when p_t itself is unknown?
 2. **Sampling:** How do we generate samples without stepping into regions where the score is undefined?



Since the data distribution p_0 is unknown, we cannot mathematically define the intermediate distributions p_t directly. Instead, we define the **process** that evolves them.

- **The Forward SDE:** We construct a diffusion process that gradually transforms data into noise:

$$dx = \underbrace{f(x, t) dt}_{\text{Drift: Deterministic change}} + \underbrace{g(t) dw}_{\text{Diffusion: Random noise}}, \quad x_0 \sim p_0(x_0)$$

- **Design the Forward SDE:** We have to **design** the SDE to satisfy two critical conditions:

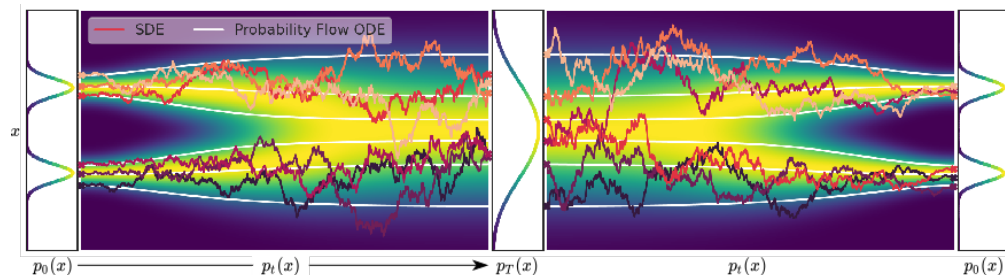
1. **Computable Transition:** The transition probability $p_{t|0}(x_t | x_0)$ must be a tractable Gaussian. This avoids simulating the SDE step-by-step during training.
2. **Simple Prior:** The process must converge to a standard Gaussian $p_T(x) \approx \mathcal{N}(0, I)$, providing a known starting point for sampling.

⇒ Because Condition 1 gives us a known $p_{t|0}(x_t | x_0)$, we can use **Denoising Score Matching** to train the score network $s_\theta(x, t)$, completely bypassing the unknown $p_t(x)$.

Once the score of the every time step $s_\theta(x, t)$ is learned, how do we sample? Naive Langevin dynamics might step into low-density regions where the score is inaccurate.

- **The Probability Flow ODE:** Remarkably, for every SDE, there exists a deterministic Ordinary Differential Equation (ODE) whose **marginal distributions $p_t(x)$ equal to those of the SDE for all t .**
- **The Sampling Mechanism:**
 1. Sample $x_T \sim p_T$ (Simple Gaussian, support everywhere).
 2. Solve the ODE backward in time from T to 0.

⇒ Since we follow a valid probability flow starting from a well-defined region, the trajectory x_t stays within the support of p_t at all times, avoiding undefined score regions.



We have established the theoretical framework (Forward SDE $\rightarrow$ Reverse ODE). Now, we will construct the actual model in three concrete steps:

- 1. Step 1: Specify the SDE (VP SDE)** Define the specific drift $f(x, t)$ and diffusion $g(t)$ to ensure convergence to Gaussian noise.
- 2. Step 2: Train the Score (Score Matching)** Train a neural network $s_{\theta(x, t)}$ to estimate the gradient field $\nabla_x \log p_{t(x)}$.
- 3. Step 3: Generate (Reverse ODE)** Plug the learned score into the Probability Flow ODE and solve it backwards from noise to data.

Step 1: Specify the SDE (VP SDE)

A standard choice for the probability flow is the **Variance Preserving (VP) SDE**, which is the continuous-time limit of DDPM.

- **The SDE Formulation:** We define the drift and diffusion using a time-dependent noise schedule $\beta(t) > 0$:

$$dx = \underbrace{-\frac{1}{2}\beta(t)x}_{\text{Drift: Shrinks signal}} dt + \underbrace{\sqrt{\beta(t)}}_{\text{Diffusion: Injects noise}} dw$$

- **Why “Variance Preserving”?** If the initial data x_0 has unit variance, the variance of x_t remains exactly 1 for all t . The drift smoothly scales down the data toward the origin, while the diffusion perfectly compensates by adding Gaussian noise.
- **The Tractable Transition Kernel:** Because this SDE is linear, the transition probability from x_0 to x_t is always a tractable Gaussian. Let $\alpha_t = \exp\left(-\int_0^t \beta(s) ds\right)$:

$$p_{t|0}(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$$

- **Practical Implementation (Linear Schedule):** In practice, we set $t \in [0, 1]$ and define $\beta(t)$ as a linear function:

$$\beta(t) = \beta_{\min} + t(\beta_{\max} - \beta_{\min})$$

Standard hyperparameters are $\beta_{\min} = 0.1$ and $\beta_{\max} = 20$.

This gives a simple, closed-form solution for code implementation:

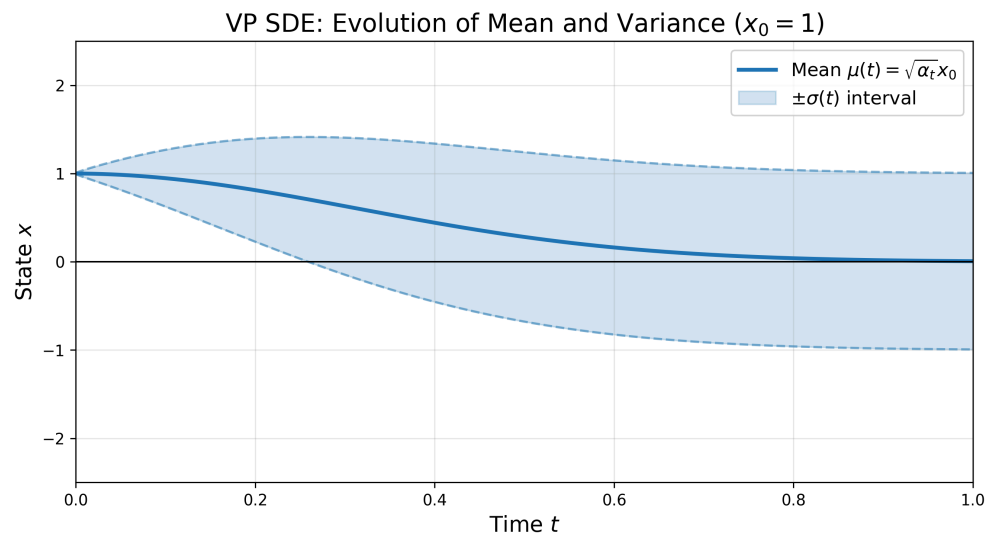
$$\begin{aligned} \int_0^t \beta(s) \, ds &= \beta_{\min} t + \frac{1}{2}(\beta_{\max} - \beta_{\min})t^2 \\ \Rightarrow \alpha_t &= \exp\left(-\beta_{\min} t - \frac{1}{2}(\beta_{\max} - \beta_{\min})t^2\right) \end{aligned}$$

$\Rightarrow$ At $t = 1$, $\alpha_1 \approx \exp(-10) \approx 4.5 \times 10^{-5} \approx 0$. The signal is completely destroyed, ensuring $p_1(x) \approx \mathcal{N}(0, I)$.

- **The Corresponding Probability Flow ODE:** By substituting our specific drift $f(x, t)$ and diffusion $g(t)$ into the general Probability Flow ODE formula, we get the following equation:

$$dx = \left(-\frac{1}{2}\beta(t)x - \frac{1}{2}\beta(t)\nabla_x \log p_t(x) \right) dt$$

- **Visualizing the Evolution ($x_0 = 1$):** How do these equations transform our data? Let's track a single 1D data point starting exactly at $x_0 = 1$ over time $t \in [0, 1]$.



- **Drift (The Mean):** The mean $\mu(t) = \sqrt{\alpha_t} x_0$ smoothly decays from 1 down to 0. The original signal is systematically erased.
- **Diffusion (The Variance):** The $\pm 2\sigma(t)$ interval, where $\sigma(t) = \sqrt{1 - \alpha_t}$, steadily expands until the distribution perfectly matches standard Gaussian noise $\mathcal{N}(0, 1)$ at $t = 1$.

Step 2: Train the Score (Score Matching)

We now have a specific SDE and its transition kernel. The task is to train a neural network $s_{\theta(x,t)}$ to approximate the score $\nabla_x \log p_t(x)$ across all continuous times $t \in [0, 1]$.

- **Extending Denoising Score Matching:** Just like in the discrete case, matching the intractable marginal score is mathematically equivalent to matching the **known** conditional score $\nabla_{x_t} \log p_{t|0}(x_t \mid x_0)$.
- **The Conditional Score of the VP SDE:** Since the transition kernel is $p_{t|0}(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$, the score is simply the gradient of the Gaussian log-density:

$$\nabla_{x_t} \log p_{t|0}(x_t \mid x_0) = -\frac{x_t - \sqrt{\alpha_t}x_0}{1 - \alpha_t}$$

Notice: This is a deterministic, closed-form target!

- **The Continuous-Time Objective:** We train our score network $s_\theta(x_t, t)$ by minimizing the expected distance to this conditional score. We sample time t uniformly, original data x_0 , and perturbed data x_t .

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0,1)} \mathbb{E}_{x_0 \sim p_0} \mathbb{E}_{x_t \sim p_{t|0}} \left[\lambda(t) \left\| s_{\theta}(x_t, t) - \underbrace{\left(-\frac{x_t - \sqrt{\alpha_t} x_0}{1 - \alpha_t} \right)}_{\nabla_{x_t} \log p_{t|0}(x_t | x_0)} \right\|_2^2 \right]$$

- **The Weighting Function $\lambda(t)$:** The score magnitude grows extremely large as $t \rightarrow 0$ (because the variance $(1 - \alpha_t) \rightarrow 0$). To prevent gradient explosion and balance the loss across all times, we typically choose a positive weighting function proportional to the variance, such as $\lambda(t) = 1 - \alpha_t$.

⇒ By optimizing this single objective, $s_\theta(x, t)$ learns the continuous vector field. We can then plug it directly into the Reverse ODE to generate data!

Step 3: Generate (Reverse ODE)

To generate data, we need to reverse the diffusion process. We utilize the fundamental theorem that connects Stochastic processes (SDE) to Deterministic processes (ODE).

Theorem 0.2 (Probability Flow ODE [3]): For any SDE $dx = f(x, t) dt + g(t) dw$, there exists a corresponding deterministic **Ordinary Differential Equation (ODE)**.

This ODE shares the **exact same marginal probability densities** $p_t(x)$ as the SDE at all times t :

$$dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt$$

- **Implication:** If we know the score $\nabla_x \log p_t(x)$, we can deterministically transport samples from the noise distribution p_T back to the data distribution p_0 .

In practice, we replace the unknown true score with our trained neural network $s_{\theta(x,t)}$.

- **The Generative Equation:** Substituting the VP SDE terms ($f = -\frac{1}{2}\beta(t)x$, $g = \sqrt{\beta(t)}$), we solve the following ODE **backwards** from $t = T$ to 0:

$$dx = \left[-\frac{1}{2}\beta(t)x - \frac{1}{2}\beta(t)s_{\theta(x,t)} \right] dt$$

- **The Sampling Steps:**
 1. **Initialize:** Sample pure noise $x_T \sim \mathcal{N}(0, I)$.
 2. **Solve:** Integrate the ODE from T to 0 using a numerical solver (e.g., Euler, Runge-Kutta, or DPM-Solver).
 3. **Output:** The final state x_0 is the generated data.

Summary: The Diffusion Framework

We have constructed a generative model based on the Probability Flow ODE.

- 1. Forward Process (VP SDE):** Designed a linear SDE that smoothly degrades data into Gaussian noise, providing a tractable transition kernel $p(x_t \mid x_0)$.
- 2. Training (Score Matching):** Learned the score function $s_{\theta(x,t)}$ by matching it against the analytic conditional score, bypassing the intractable marginal likelihood.
- 3. Generation (Reverse ODE):** Generated new data by solving the deterministic ODE backwards, using the learned score to navigate from noise p_T back to the data manifold p_0 .

Controllable Generation: Guiding the SDE

We now know how to generate unconditional data $x \sim p_0(x)$. What if we want to generate data conditioned on a signal y (e.g., a text prompt or a class label)?

- **The Conditional Score:** To generate conditional samples, we simply replace our unconditional score $\nabla_x \log p_t(x)$ with the conditional score $\nabla_x \log p_t(x | y)$ inside the Reverse ODE/SDE.
- **Applying Bayes' Theorem:** By taking the gradient of the log-posterior $p_t(x | y) \propto p_t(x)p_t(y | x)$, we get a beautiful decomposition:

$$\nabla_x \log p_t(x | y) = \underbrace{\nabla_x \log p_t(x)}_{\substack{\text{Unconditional Score} \\ s_\theta(x,t)}} + \underbrace{\nabla_x \log p_t(y | x)}_{\substack{\text{Guidance Term} \\ \text{(Likelihood)}}}$$

- **The Mechanism:**
 1. **Unconditional Score:** Ensures the generated sample looks like real, high-quality data.
 2. **Guidance Term:** Acts as a secondary vector field, actively pushing the trajectory toward regions of the data manifold that satisfy the condition y .

In **Diffusion Planner**, we want to generate valid trajectories that satisfy specific physical or operational constraints (e.g., avoiding obstacles, target speeds).

- **Rethinking the Guidance Term:** Instead of training a separate neural network classifier to predict $p_t(y \mid x)$, we can mathematically define these rules using **human-designed cost functions** $\mathcal{E}(x)$.
- **Energy-Based Formulation:** We formulate the condition y (“satisfies all constraints”) as a Boltzmann distribution. The probability of a trajectory being valid decreases exponentially as the cost increases:

$$p_t(y \mid x_t) \propto \exp(-\lambda \mathcal{E}(x_t))$$

where $\lambda > 0$ controls the guidance strength, and $\mathcal{E}(x_t)$ is the total cost.

- **Analytic Guidance Field:** By plugging this into our Bayes' decomposition, the intractable log-probability derivative beautifully simplifies into an **analytic gradient**:

$$\nabla_{x_t} \log p_t(y \mid x_t) = -\lambda \nabla_{x_t} \mathcal{E}(x_t)$$

(e.g., $\mathcal{E}(x_t) = \mathcal{E}_{\text{obstacle}}(x_t) + \mathcal{E}_{\text{velocity}}(x_t)$)

⇒ **The Synergy:** The unconditional score $s_\theta(x_t, t)$ ensures the trajectory follows realistic dynamics (learned from data), while $-\lambda \nabla_{x_t} \mathcal{E}(x_t)$ acts as a “repulsive force” pushing the path away from obstacles during inference!

Advanced Capabilities Enabled by the ODE Formulation

Because the Probability Flow ODE is completely **deterministic**, it defines a bijective (1-to-1) mapping between the complex data space $x_0 \sim p_0$ and the simple latent space $x_T \sim p_T$.

- **How to Interpolate:**

1. **Encode:** Take two real images $x_0^{(A)}$ and $x_0^{(B)}$. Simulate the ODE **forward** ($t = 0 \rightarrow T$) to find their exact latent representations $x_T^{(A)}$ and $x_T^{(B)}$.
 2. **Interpolate:** Blend the latents in the spherical prior space using a parameter $\lambda \in [0, 1]$ (e.g., $z_\lambda = \text{Slerp}(x_T^{(A)}, x_T^{(B)}, \lambda)$).
 3. **Decode:** Simulate the ODE **backward** ($t = T \rightarrow 0$) starting from z_λ to generate the intermediate image $x_0^{(\lambda)}$.
- **Result:** Unlike pixel-space blending, ODE-based interpolation smoothly morphs the semantic features (e.g., age, hair color) while staying perfectly on the data manifold.

Application 1: Latent Space Interpolation

38 / 44



The Reverse ODE allows us to compute the **exact** probability density $p_0(x)$ of any data point.

- **Instantaneous Change of Variables:** Let $f_{\text{ODE}}(x, t)$ be the vector field of our Probability Flow ODE. The change in the log-probability over time is given by the negative divergence of this vector field:

$$\frac{\partial \log p_t(x)}{\partial t} = -\nabla_x \cdot f_{\text{ODE}}(x, t)$$

- **Computing the Likelihood:** To find the exact log-likelihood of a real image x_0 , we integrate this divergence along the ODE trajectory from $t = 0$ to $t = T$:

$$\log p_0(x_0) = \log p_T(x_T) + \int_0^T \nabla_{x_t} \cdot f_{\text{ODE}}(x_t, t) dt$$

where $p_T(x_T)$ is just the simple Gaussian prior $\mathcal{N}(0, I)$.

- **Why this matters:** Exact likelihoods allow for principled model evaluation, rigorous comparison against other generative models (like Autoregressive models or Normalizing Flows), and out-of-distribution (anomaly) detection.

Accelerating Generation: DPM-Solver++

While the Probability Flow ODE is elegant, solving it with standard numerical methods (like Euler or Runge-Kutta) is notoriously slow, often requiring 200 to 1000 steps.

- **Why so slow?** The ODE contains a “stiff” linear drift term. If we take large step sizes, the truncation error explodes, destroying the image.

$$dx = \underbrace{-\frac{1}{2}\beta(t)x}_{\text{Linear Drift (Stiff)}} dt - \frac{1}{2}\beta(t) \underbrace{\varepsilon_{\theta}(x, t)}_{\text{Neural Network}} dt$$

- **The Semi-Analytic Solution (Exponential Integrator):** Instead of treating the whole ODE as a black box, we exploit its semi-linear structure. By applying the variation of constants formula, we can solve the linear part $-\frac{1}{2}\beta(t)x$ **exactly analytically**.

$$x(t) = \underbrace{e^{-\int_s^t \frac{1}{2}\beta(r) dr} x(s)}_{\text{Exact Linear Evolution}} - \int_s^t e^{-\int_u^t \frac{1}{2}\beta(r) dr} \frac{1}{2}\beta(u)\varepsilon_\theta(x_u, u) du$$

⇒ We only need to numerically approximate the integral of the neural network part ε_θ . Since ε_θ (or data prediction x_θ) changes much more smoothly than the raw drift, we can take huge steps with minimal error!

References

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>
- [2] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics,” in Proceedings of the 32nd International Conference on Machine Learning (ICML), 2015. [Online]. Available: <https://proceedings.mlr.press/v37/sohl-dickstein15.html>
- [3] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,”

in International Conference on Learning Representations (ICLR), 2021. [Online]. Available: <https://openreview.net/forum?id=PxTIG12RRHS>

- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” in International Conference on Learning Representations (ICLR), 2023. [Online]. Available: <https://openreview.net/forum?id=PqvMRDCJT9t>
- [5] P. Vincent, “A Connection Between Score Matching and Denoising Autoencoders,” Neural Computation, 2011, doi: [10.1162/NECO_a_00142](https://doi.org/10.1162/NECO_a_00142).
- [6] Y. Song and S. Ermon, “Generative Modeling by Estimating Gradients of the Data Distribution,” in Advances in Neural Information Processing Systems (NeurIPS), 2019. [Online]. Available: <https://papers.nips.cc/paper/9361-generative-modeling-by-estimating-gradients-of-the-data-distribution>